



Laboratory 1

Understanding Assembly

Assembly language is the human-readable representation of the computer's native language. Each assembly language instruction specifies both the operation to perform and the operands on which to operate. Assembly languages were originally designed with a one-to-one correspondence between mnemonics and machine language instructions. Translating from mnemonics to machine language became the job of a systems program known as an *assembler*. Assemblers were eventually augmented with elaborate "macro expansion" facilities to permit programmers to define parameterized abbreviations for common sequences of instructions. The correspondence between assembly language and machine language remained obvious and explicit, however. Programming continued to be a machine-centered enterprise: each different kind of computer had to be programmed in its own assembly language, and programmers thought in terms of the instructions that the machine would actually execute.

Assembly language is a key component in creating effective *shellcode*. High-level programming languages generate code that contains all kinds of data that should not be in shellcode. With assembly language, every instruction is translated literally in executable bits that the processor understands. Choosing the correct shellcode to compromise a host can often determine the success of an attack. Depending on the shellcode used by the attacker, the exploit is far more (or less) likely to be detected by a network- or host-based intrusion detection system/intrusion prevention system.

Data stored on the stack can end up overwriting beyond the end of the allocated space and thus overwrite values in the register, changing the execution path as a result. Changing the execution path to point to the payload sent can help execute commands.

Security vulnerabilities related to buffer overflows are the largest share of vulnerabilities in the information security industry. Even though software vulnerabilities that result in stack overflows are not as common these days, they are still found in software. With the knowledge of stack overflows and the understanding of how to write exploits, one should be armed enough to write exploits.

The goal of any exploit is always to take control of EIP (current instruction pointer) and point it to the malicious or shellcode sent by the exploit to execute a command on the system. Techniques such as XOR or bit-flipping can be used to avoid problems with NULL bytes. To stabilize code and to make it work across multiple versions of operating systems, an exception handler can be used to automatically detect the version and respond with appropriate shellcode.



1. Preparation:

- a. You can use any debugger/disassembler as long as it supports the given task architecture (32bit or 64bit)
- b. Try to stay away from decompilers, as this will help you to better understand the nature of your task based on assembly only, remember in real-world tasks you will have to deal with much larger binaries. (N.B. Please check what each binary does before running it – do not simply trust us!)
- c. Check some write-ups about some CTF to see what you should/shouldn't include in your report
- d. Try to do the lab in a Linux VM, as you might need to disable ASLR.

2. Theory

- a. What is ASLR, and why do we need it?
- b. What kind of file did you receive (which architecture (32-bit or 64-bit?))?
- c. What do stripped binaries mean?
- d. What are GOT and PLT?
- e. How can the debugger insert a breakpoint in the debugged binary/application?

3. Disassembly

- a. Disable ASLR using the following command `sudo sysctl -w kernel.randomize_va_space=0`
- b. Load the binaries **from the Task 3 folder** into a disassembler/debugger
- c. Do the function prologue and epilogue differ in 32-bit and 64-bit?
- d. Do function calls differ in 32-bit and 64-bit? What about argument passing?
- e. What does the command **ldd** do? `ldd BINARY-NAME`
- f. Why in the "sample64-2" binary, the value of **i** didn't change even if our input was very long?

4. Reversing

- a. You will have multiple binaries **in the Task 4 folder**, Try to reverse them by recreating them using any programming language of your choice (C is preferred)



Notes:

- Make the report as technical as possible (no installation guide please).
- ALWAYS include a PoC code with your report (if required)
- Try to have assembly code in a formatted block (check markdown)
- If you paste some data (command output), make sure it is readable and the format did not change
- If you want to include a command in the report, please highlight it (bold, italic, different layout, ...)
- The compile command for each file is the following:
 - `sample64 = gcc -fno-stack-protector -g -o sample64 sample.c`
 - `sample64-2 = gcc -o sample64-2 sample.c`
 - `sample32 = gcc -m32 -fno-stack-protector -g -o sample32 sample.c`